

Bandit SAST Summary

Security Report | Python SAST

Author: Jagriti Banerjee

Project: Enterprise DevSecOps Security Lab

Document Type: Bandit SAST Summary

This report summarizes the Bandit static application security testing results for the Python Flask component of the Enterprise DevSecOps Security Lab. It presents scan scope, methodology, risk interpretation, remediation guidance, CI/CD relevance, and validation criteria in a professional security-reporting format.

Total Findings 4	High 1	Medium 2	Low 1
---------------------	-----------	-------------	----------

Assessment Focus Python Flask application SAST Subprocess, SQL query, and runtime binding review	Evidence Sources Bandit JSON report Local scan screenshot CI/CD job output	Outcome 4 findings reviewed 1 High, 2 Medium, 1 Low Remediation plan included
---	---	--

Private Lab Evidence - No third-party systems assessed

1. Executive Overview

Bandit identified four security-relevant observations in `src/app/app.py`. The results align with the deliberately vulnerable design of this private lab: command execution and SQL query construction were intentionally implemented in an unsafe way to demonstrate detection, reporting, remediation, and pipeline integration. The highest-risk issue is command injection caused by `shell=True` in a subprocess call.

ID	Bandit Test	Severity	Confidence	Short Description
BANDIT-001	B404 - blacklist	LOW	HIGH	Consider possible security implications associated with the subprocess module.
BANDIT-002	B608 - hardcoded_sql_expressions	MEDIUM	LOW	Possible SQL injection vector through string-based query construction.
BANDIT-003	B602 - subprocess_popen_with_shell_equals_true	HIGH	HIGH	subprocess call with <code>shell=True</code> identified, security issue.
BANDIT-004	B104 - hardcoded_bind_all_interfaces	MEDIUM	MEDIUM	Possible binding to all interfaces.

2. Scope and Scan Context

Scope Item	Details
Application scanned	Python Flask application in <code>src/app/</code>
Primary file with findings	<code>src/app/app.py</code>
Supporting file scanned	<code>src/app/init_db.py</code>
Testing category	Static Application Security Testing (SAST)
Evidence source	<code>security/reports/bandit-report.json</code> , local terminal output, GitHub Actions output
Lab classification	Private educational DevSecOps lab; no third-party targets assessed

3. Bandit Methodology

Bandit was used to scan Python source code for insecure implementation patterns. The assessment reviewed each finding in context instead of treating raw scanner output as the final result. Each observation was mapped to its route-level exposure, exploit scenario, remediation direction, and CI/CD validation requirement.

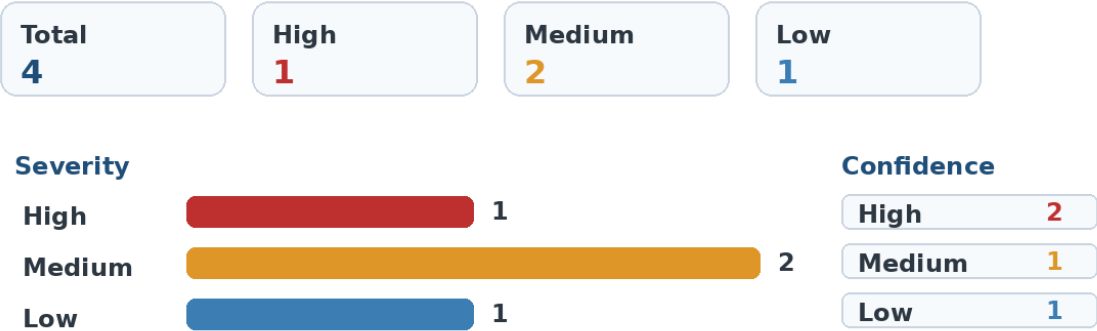
- Collected the Python application source from `src/app`.
- Executed Bandit recursively against the Flask application codebase.
- Exported structured JSON output to `security/reports/bandit-report.json`.
- Reviewed each issue by file, line, Bandit ID, severity, and confidence.
- Mapped findings to command injection, SQL injection, subprocess usage, and service exposure risks.
- Documented remediation guidance and production gating recommendations.

4. Scan Result Metrics

The following metrics summarize the Bandit output. The finding count is small because the application is intentionally compact, but the issues represent high-value SAST learning outcomes: command execution safety, SQL query safety, and local-development exposure control.

Bandit SAST Finding Distribution

Severity and confidence overview generated from security/reports/bandit-report.json



Files scanned: 2 | Lines of code: 70 | Skipped tests: 0

Figure 1: Bandit finding distribution by severity and confidence.

Metric	Value	Interpretation	Recommended Action
Total findings	4	Four Bandit observations were identified.	Review all findings; prioritize command injection first.
High severity	1	One high-risk command execution pattern.	Immediate remediation recommended.
Medium severity	2	SQL string construction and network bind warning.	Fix before production deployment.
Low severity	1	Subprocess import warning.	Track with B602 remediation.
High confidence	2	Two findings are highly reliable.	Treat as validated unless code context disproves.
Skipped tests	0	No tests skipped.	Scan coverage acceptable for lab scope.

5. Route-Level Risk Mapping

Route / Component	Finding Link	Attack Scenario	Primary Risk	Remediation Theme
/ping?host=	B602, B404	Append shell metacharacters to host input.	Command injection inside container.	Use shell=False and strict allowlisting.
/user?id=	B608	Manipulate id parameter to alter SQL WHERE clause.	Unauthorized row retrieval.	Parameterized queries and type validation.
__main__ app.run	B104	Service listens on all interfaces during direct execution.	Unintended exposure of debug/test service.	Production WSGI server and controlled binding.

6. Detailed Findings

Each finding below is documented with practical risk interpretation and remediation direction. This makes the report useful as more than scanner output: it supports secure-code review, Jira-style remediation, and retest planning.

6.1 BANDIT-001: B404 - blacklist

File / Line	Severity	Confidence	Bandit Message
app.py:3	LOW	HIGH	Consider possible security implications associated with the subprocess module.

Priority context: Contextual warning. The import is not exploitable by itself but is relevant because subprocess is later used unsafely.

Risk interpretation: Unsafe subprocess usage can create command injection when input is passed through a shell.

Remediation guidance: Keep subprocess only if required; use shell=False, fixed binaries, strict validation, and timeouts.

Relevant code excerpt:

```
2 import sqlite3
3 import subprocess
4 import os
```

Example secure direction:

```
subprocess.run(["ping", "-c", "1", host], shell=False, timeout=5)
```

6.2 BANDIT-002: B608 - hardcoded_sql_expressions

File / Line	Severity	Confidence	Bandit Message
app.py:38	MEDIUM	LOW	Possible SQL injection vector through string-based query construction.

Priority context: High secure-coding priority. User input is embedded directly into a SQL query string.

Risk interpretation: An attacker can manipulate the id parameter to alter query logic and retrieve rows outside the intended record.

Remediation guidance: Use parameterized queries, type validation for numeric IDs, and avoid returning raw SQL query strings to clients.

Relevant code excerpt:

```
37
38     query = f"SELECT id, username, role FROM users WHERE id = '{user_id}'"
39
```

Example secure direction:

```
cursor.execute("SELECT id, username, role FROM users WHERE id = ?", (user_id,))
```

6.3 BANDIT-003: B602 - subprocess_popen_with_shell_equals_true

File / Line	Severity	Confidence	Bandit Message
app.py:55	HIGH	HIGH	subprocess call with shell=True identified, security issue.

Priority context: High priority. User-controlled input is passed into a shell command with shell=True.

Risk interpretation: A crafted host parameter can append shell metacharacters and execute additional commands inside the container. In a weaker runtime environment, this could enable reconnaissance, file access, or lateral movement attempts.

Remediation guidance: Replace shell=True with argument-list execution, validate the host input with strict allowlisting, reject shell metacharacters, and add a timeout.

Relevant code excerpt:

```

54         f"ping -c 1 {host}",
55         shell=True,
56         capture_output=True,
57         text=True
58     )
59
60     return f"<pre>{result.stdout}\n{result.stderr}</pre>"
61

```

Example secure direction:

```

subprocess.run(["ping", "-c", "1", host], shell=False, capture_output=True, text=True,
               timeout=5)

```

6.4 BANDIT-004: B104 - hardcoded_bind_all_interfaces

File / Line	Severity	Confidence	Bandit Message
app.py:73	MEDIUM	MEDIUM	Possible binding to all interfaces.

Priority context: Medium operational security concern. Binding to all interfaces can expose the service beyond the intended local test scope.

Risk interpretation: If a development server or debugging configuration is reachable from untrusted networks, attackers may access vulnerable routes.

Remediation guidance: Avoid Flask development server in production, bind locally during testing, and use Unicorn plus Kubernetes Service/Ingress controls.

Relevant code excerpt:

```

72     debug_mode = os.getenv("FLASK_DEBUG", "false").lower() == "true"
73     app.run(host="0.0.0.0", port=5000, debug=debug_mode)

```

Example secure direction:

```

app.run(host="127.0.0.1", port=5000, debug=False)

```

7. Recommended Remediation Plan

Priority	Finding	Owner Activity	Recommended Fix	Validation
P1	B602 shell=True command injection	Secure /ping route	Replace shell=True, validate host, add timeout	B602 removed; injection test fails safely
P2	B608 SQL string construction	Secure /user lookup	Use parameterized query and numeric ID validation	B608 removed; SQL injection returns no extra rows
P3	B104 bind all interfaces	Harden runtime mode	Use Gunicorn in containers and avoid dev server in production	Risk removed or documented as local-only
P4	B404 subprocess import	Review subprocess usage	Keep only safe subprocess patterns	No unsafe subprocess usage remains

8. CI/CD Integration Summary

Bandit was integrated into the GitHub Actions CI/CD workflow as the SAST stage. This makes application security testing repeatable on repository changes and pull requests. In this lab, the job preserves output for evidence while allowing downstream DevSecOps stages to continue for demonstration.

- Bandit runs as part of the Application Security Testing job.
- JSON output is generated for structured reporting and artifact retention.
- Terminal output is captured as pipeline evidence for reviewers.
- For production gating, HIGH severity findings should fail the build unless risk-accepted.

9. Production Gating Recommendation

The lab currently demonstrates findings without blocking later pipeline stages. For an enterprise pipeline, Bandit should become a merge gate on production-bound branches.

- Fail the CI job for HIGH severity findings with MEDIUM or HIGH confidence.
- Fail immediately for command execution, deserialization, hardcoded secret, or credential exposure findings unless formally risk-accepted.
- Allow MEDIUM findings only with an owner, ticket, and target remediation date.
- Store Bandit ISON and summarized output as workflow artifacts.

```
bandit -r src/app -f json -o security/reports/bandit-report.json --severity-level high
--confidence-level medium
```

10. Retest and Validation Checklist

Validation Item	Expected Result	Evidence to Capture
Re-run Bandit recursively on src/app	B602 and B608 removed after code fixes	Updated bandit-report.json and terminal screenshot
Execute command injection payload against /ping	Payload is rejected or treated as invalid input	Curl output and application logs
Execute SQL injection payload against /user	Injection does not alter query results	Curl output and application logs
Run CI/CD pipeline	SAST stage passes or fails according to policy	GitHub Actions screenshot
Review Docker/Kubernetes hardening	Runtime remains non-root and read-only	securityContext evidence

11. Evidence Exhibits

The following evidence screenshots support the Bandit assessment. They are intentionally placed after the analysis sections to prevent crowding the cover page and to keep the report flow clean.



Exhibit A: Local Bandit SAST findings against the Flask application.

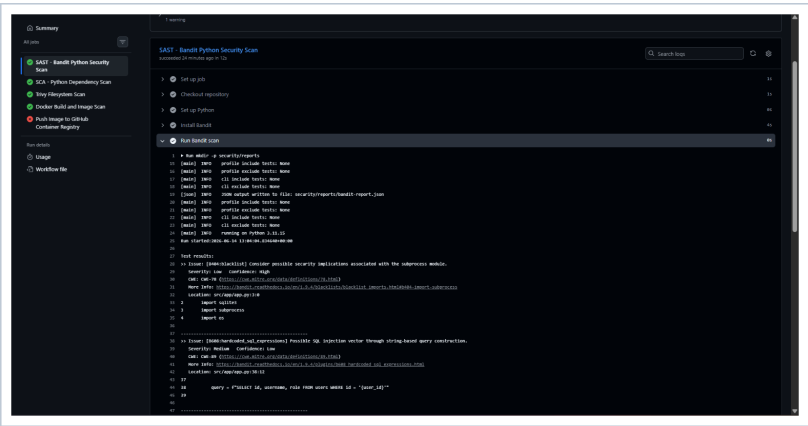


Exhibit B: GitHub Actions Bandit job output showing CI/CD SAST evidence.

12. Conclusion

The Bandit assessment successfully identified the intentionally vulnerable patterns embedded in the Flask application. The results demonstrate SAST execution, secure code review reasoning, CI/CD security integration, remediation planning, and professional reporting. The highest-priority fix is to remove `shell=True` from the `/ping` route, followed by parameterizing SQL queries in the `/user` route. After remediation, Bandit should be re-run and the updated report retained as evidence of risk reduction.

Appendix A: Raw Finding Metadata

Test ID	Test Name	File	Line	Severity	Confidence
B404	blacklist	app.py	3	LOW	HIGH
B608	hardcoded_sql_expressions	app.py	38	MEDIUM	LOW
B602	subprocess_popen_with_shell_equals_true	app.py	55	HIGH	HIGH
B104	hardcoded_bind_all_interfaces	app.py	73	MEDIUM	MEDIUM

Appendix B: Secure Coding Reference

Risk Area	Unsafe Pattern	Secure Direction
Command execution	<code>subprocess.run(f"ping -c 1 {host}", shell=True)</code>	Use <code>shell=False</code> , argument lists, strict allowlisting, and command timeout.
SQL query construction	<code>query = f"SELECT ... WHERE id = {user_id}"</code>	Use parameterized queries and numeric ID validation.
Service exposure	<code>app.run(host="0.0.0.0", debug=True)</code>	Use Gunicorn in containers and bind through Kubernetes Service/Ingress controls.
CI/CD gating	Security scan allowed to pass silently	Fail production-bound builds for HIGH severity findings unless risk-accepted.

Appendix C: Bandit Command Reference

```
bandit -r src/app -f json -o security/reports/bandit-report.json || true
bandit -r src/app
bandit -r src/app --severity-level high --confidence-level medium
```